Ders 13 Çalışma Özeti

Ders 13 Çalışma Özeti

Genel Konular

- Dersin tarihi 21 Nisan 2021 Çarşamba. Ders Zoom üzerinden yapıldı.
  - Asıl ders yaklaşık 00:04'te başlar. İki ara vardır: yaklaşık 00:50-01:04 ve 01:50-02:04.
- Dersin başındaki konuşma
  - Derslere katılım düşük. Hoca devam almıyor, ama "ders derste öğrenilir" dedi. Etkileşim olmazsa dersin anlamı azalır.
  - Bir öğrenci neden hep dizi kullanıldığını, `ArrayList` kullanılıp kullanılamayacağını sordu. Hoca generic'lere gelince göreceklerini, ama bilenlerin laboratuvarda kullanabileceğini söyledi.
- Java'da olmayan nesne yönelimli kavramlar
  - Copy constructor (daha önce konuşuldu), operatör aşırı yükleme (operator overloading) ve sınıflar arası çoklu kalıtım.
- Operatör aşırı yükleme
  - Java'da `+` yalnızca ilkel tiplerde ve `String` birleştirmede çalışır. `Complex c3 = c1 + c2;` derleme hatası verir: "bad operand types for binary operator '+'".
  - Java'da çözüm metot yazmaktır. Üç yol gösterildi: statik `add(c1, c2)`, nesne metodu `c1.add(c2)` ve nesnenin kendisini değiştiren `add`.
  - C#'ta `public static MyComplex operator +(MyComplex c1, MyComplex c2)` ile `+` tanımlanır.
  - C#'ta `==` aşırı yüklenirse `!=` de tanımlanmak zorundadır.
  - C++'ta operatör, sınıfın üye fonksiyonu olarak `this` üzerinden tanımlanabilir. Python'da `__add__`, `__lt__` gibi özel metotlar kullanılır.
  - Yeni bir operatör (ör. `K` harfi) tanımlanamaz. Yalnızca dilin mevcut operatörleri aşırı yüklenebilir. Bazı operatörler hiç aşırı yüklenemez.
- Çoklu kalıtım
  - Python'da `class MultiDerived(Base1, Base2)` yazılabilir. C++'ta da vardır.
  - Java'da çok seviyeli kalıtım (multilevel) vardır, ama bir sınıf iki üst sınıftan kalıtım alamaz. Bunun yerine birden fazla interface implement edilir.
- `==` ile `equals` farkı
  - Nesnelerde `c1 == c2` referansları karşılaştırır. İçerik aynı olsa da farklı nesnelerse `false` döner.
  - İçeriği karşılaştırmak için `equals` override edilir. NetBeans'in "Insert Code" ile ürettiği `equals` gösterildi.
- Interface'ler
  - Bir öğrenci Swift'teki protokolleri örnek verdi.
  - Gündelik anlam: Arka plandaki ayrıntıyı bilmeden bir şeyi kullanmayı sağlayan yüz. Örnekler: monitörün tuşları, grafik kullanıcı arayüzü (GUI), banka sistemi.
  - Java'da interface, alanı olmayan, yalnızca metot imzaları içeren bir yapıdır. Yalnızca `public static final` sabitler tanımlanabilir. Yeni Java sürümlerinde `default` metotlar da yazılabilir.
  - UML'de `<<interface>>` stereotipiyle gösterilir. Kalıtım düz okla, interface gerçekleştirme kesikli okla gösterilir.
  - Bir sınıf `implements` ile birden fazla interface alabilir. Sınıf, interface'teki bütün metotları yazmak zorundadır. Yazmazsa derleme hatası alır.
  - Kullanım amaçları: sorumlulukları gruplamak (`IPrinter`: print, updateDriver), nesneye birden fazla görünüm vermek, ağır olan kalıtım yerine hafif bir ilişki kurmak.

  - Kurallar: Normal üye alan yok. Constructor yok. Interface'ten nesne üretilmez, yalnızca referans alınır. İsimler genellikle `I` ile başlar (`IDatabase`).
  - Interface'ler arasında da kalıtım olabilir. Tekrar eden bir metot (`getPlate`) üst interface'e çıkarılabilir.
- Slayt örneği: Araç vergisi
  - `ICommercialVehicle` ve `IPersonalVehicle` interface'leri. Motosiklet ve araba her ikisini, otobüs yalnızca ticari olanı implement eder.
  - Yalnızca ticari araçların vergisi amortize edilebilir (`calculateAmortizedTax`).
  - Vergi hesapları birbirine benziyor olsaydı soyut sınıf daha iyi bir seçim olurdu. Farklı oldukları için interface seçildi.
  - `VehicleRegistrationSystem` sınıfı ticari ve kişisel araçları iki ayrı listede tutar.
- Kod örneği: `IDatabase`
  - Metotlar: `Book search(String name)`, `int delete(String name)`, `void insert(Book b)`.
  - `OracleDB` ve `MsSqlDatabase` bu interface'i implement eder.
  - `IDatabase r = new OracleDB();` yazılır. Veritabanı değişince yalnızca bu satır `new MsSqlDatabase()` olur. Geri kalan kod aynı kalır.
  - İleride thread'lerde `Runnable` interface'i kullanılacak. Onun `run` metodunu yazmak zorunlu olacak.
- Enum'lar
  - Sınırlı bir değer kümesi için kullanılır: beden (S, M, L, XL), pizza boyu.
  - Bu değerleri `int` ile kodlamak hataya açıktır. 5 veya 0 gibi geçersiz değerler girilebilir.
  - `public enum Size { SMALL, MEDIUM, LARGE, EXTRA_LARGE }`. Enum aslında bir sınıftır. Her sabit bu sınıfın bir örneğidir. Başka örnek üretilemez.
  - Enum'un alanları, metotları ve constructor'ı olabilir. Constructor private olmalıdır.
  - Örnek: `Tariff` enum'u ve `Person`, `Subscription` sınıfları.
- Generic sınıflar
  - Aynı kodu farklı veri tipleri için tekrar kullanmayı sağlar. 500 overload yazmak yerine tek bir generic metot veya sınıf yazılır. C++'taki template'lere benzer.
  - Tip parametresi açılı parantez içinde yazılır: `class GenericClass<T>`. Yaygın harfler: `T` (type), `E` (element), `K` (key), `V` (value), `N` (number).
  - İlkel tipler tip parametresi olamaz. `GenericClass<int>` hata verir. Sarmalayıcı (wrapper) sınıf kullanılır: `GenericClass<Integer>`.
  - Wildcard'lar (`? extends`, `? super`) bu ders kapsamında değil.
- Veri yapıları ve koleksiyonlar
  - Veri yapısı: verileri bellekte düzenleme şeması. Doğru veri yapısı programın performansını belirler. Dengeli ikili ağaçta arama $O(\log n)$'dir.
  - Bağlı liste: Düğümler kendine referans tutan sınıf nesneleridir (`Node next`). Java'da işaretçi yıldızı yoktur, referans kullanılır. İlk düğüm head, son düğüm tail'dir.
  - Bağlı listenin avantajı: Ekleme ve silmede kaydırma yoktur. Dizinin avantajı: İndeksle doğrudan erişim.
  - `ArrayList`: Generic ile eleman tipi belirtilir (`ArrayList<Person>`). Metotlar: `add`, `get(i)`, `size`, `set(i, e)`, `remove(i)`, `ensureCapacity`, `trimToSize`.
  - Size ve kapasite farklıdır. Kapasite dolunca yaklaşık 1,5 katına çıkar.
  - Sınıf hiyerarşisi: `ArrayList`, `Vector` ve `LinkedList` `AbstractList` ailesindendir. `AbstractList`, `List` interface'ini, `List` de `Collection` interface'ini kullanır. `Collection`'daki `add`, `clear`, `contains`, `isEmpty`, `iterator`, `remove`, `size` metotları bu yüzden bütün listelerde vardır.
  - `Vector` eskimiş (obsolete) bir koleksiyondur. Eskimiş metotlar IDE'de üstü çizili görünür.
  - Örnek: `Course` ve `Student` (Yunus Hoca'nın kodu). Kurs, öğrencileri `ArrayList<Student>` içinde tutar. Dizideki manuel kapasite artırma işi `ArrayList`'te gerekmez.
- Sınav ve laboratuvar duyuruları
  - Laboratuvar bugün saat 21:00'de. Toprak Hoca bir Zoom linki paylaştı.
  - Kodunuzu yüklediğinizi gösteren ekran görüntüsü alın. "Yükledim ama olmadı" mazereti kabul edilmiyor. E-postayla gelen kodlar kabul edilmedi.
  - Vize: Test olmayacak. Tamamen program yazma (implementation) olacak.
    * Bir UML şeması verilip kodlamanız istenebilir. Şemada gizli bilgi (hidden infor-

mation) olabilir.
    * Basit bir senaryo verilip önce UML çizmeniz, sonra koda geçmeniz istenebilir.
    * Java kodları laboratuvardaki gibi sisteme yüklenecek. Benzerlik kontrolü yapılacak.
  - Kapsam: Soyut sınıflar ve interface'ler dahil, muhtemelen enum da dahil. Generic'ler ve veri yapıları muhtemelen dahil değil. Map'lerin ötesi kesinlikle sorulmayacak.

Hocanın Özellikle Vurguladığı Kısımlar

- **Nesnelerde == referansları karşılaştırır.** Bu, öğrencilerin çok yaptığı bir hatadır. İçerik için `equals` yazılır.
  - `c2 = c1;` yapılırsa ikisi aynı nesneyi gösterir ve `c1 == c2` `true` olur. Eski nesne daha sonra çöp toplayıcıyla silinir.
- **Overload ile override'ı karıştırmayın.** Operatör aşırı yükleme bir overload'dur.
  - Override kalıtımın olduğu yerde olur: soyut sınıftan kalıtım alırken, interface implement ederken veya normal bir sınıfın metodunu geçersiz kılarken.
- **Interface'i implement eden sınıf, interface'teki bütün metotları yazmak zorundadır.** Bu bir standart sağlar.
- Interface ile kalıtım farkı: `extends` ile yalnızca bir sınıf, `implements` ile birden fazla interface alınabilir.
- Veritabanı örneğinde asıl kazanç taşınabilirliktir. Arka plandaki teknolojiyi değiştirmek için yalnızca nesnenin üretildiği satır değişir.
- Bir işi tersinden bilmek, yani dilde neyin olmadığını bilmek de önemlidir.
- Veri yapıları dersini almayanlar bağlı liste, çift bağlı liste ve ağaç konularına bakmalı. Bu derste veri yapılarının tasarımı değil, Java'da kullanımı önemli.
- `ArrayList`'in metotlarını ve sınıf hiyerarşisini bilmek zorundasınız.
- Önerilen kaynak: javaTpoint sitesinin Java Collections bölümü.

Kısa Tekrar Notları

- Java'da operatör aşırı yükleme yok. Yerine `add` gibi metotlar yazılır.
- C#: `public static T operator +(T a, T b)`. == tanımlanırsa `!=` de tanımlanmalı.
- Python: `__add__`, `__lt__`, `__le__`, `__gt__`, `__ge__`.
- == referans, `equals` içerik karşılaştırır.
- Interface: yalnızca metot imzaları ve `public static final` sabitler. Constructor yok. Nesnesi üretilmez.
- `class A extends B implements I1, I2`.
- Enum bir sınıftır. Sabitler ilk satırda yazılır. Constructor private'tır.
- Generic: `class Kutu<T>`, `new Kutu<Integer>(5)`. İlkel tip kullanılamaz.
- `ArrayList`: size eleman sayısı, kapasite ayrılmış yer. Sınır dışı erişim `IndexOutOfBoundsException` verir.
- `ArrayList` hızlı erişim, `LinkedList` hızlı ekleme ve silme.

Detaylı Açıklamalar

Java'da karmaşık sayı toplama

```java
public class Complex {
    private int real, imaginary;
    public Complex() { this(0, 0); }
    public Complex(int real, int imaginary) { this.real = real; this.imaginary = imaginary; }

    // 1. çözüm: statik metot
    public static Complex add(Complex c1, Complex c2) {
        Complex c = new Complex();
        c.real = c1.real + c2.real;            // private alana aynı sınıfın içinden erişilebi
        c.imaginary = c1.imaginary + c2.imaginary;
        return c;
    }
```

```
    // 2. çözüm: nesne metodu, yeni nesne döndürür
    public Complex add(Complex c1) {
        Complex c = new Complex();
        c.real = this.real + c1.real;
        c.imaginary = this.imaginary + c1.imaginary;
        return c;
    }

    @Override
    public String toString() { return "Complex{" + real + " + " + imaginary + "i}"; }
}

Complex c1 = new Complex(1, 2), c2 = new Complex(2, 2);
// Complex c3 = c1 + c2;      // derleme hatası
Complex c3 = Complex.add(c1, c2);  // 3 + 4i
Complex c4 = c1.add(c2);           // 3 + 4i
Complex c5 = c2.add(c1);           // 3 + 4i, ama c4'ten farklı bir nesne
```

- Üçüncü yol, nesnenin kendisini değiştirmektir: `this.real += c1.real; this.imaginary += c1.imaginary; return this;`
  - Bu durumda orijinal nesne değişir. Derste `c1.add(c2)` sonrası `c1` 1+2i'den 3+4i'ye döndü. Sonraki çağrılar bu yeni değerle devam etti.

C#'ta operatör tanımlama

```
class MyComplex {
    public int real, imaginary;
    public MyComplex(int real = 0, int imaginary = 0) { this.real = real; this.imaginary = ima
    public static MyComplex operator +(MyComplex c1, MyComplex c2) {
        return new MyComplex(c1.real + c2.real, c1.imaginary + c2.imaginary);
    }
    public static bool operator ==(MyComplex c1, MyComplex c2) {
        return c1.real == c2.real && c1.imaginary == c2.imaginary;
    }
    public static bool operator !=(MyComplex c1, MyComplex c2) { return !(c1 == c2); }
    public override string ToString() { return real + " + " + imaginary + "i"; }
}
```

- Operatör metodu `public static` olmak zorundadır ve `operator` anahtar kelimesiyle yazılır.
- C#'ta `ToString` override edilirken `override` bir anahtar kelimedir. Java'daki gibi bir annotation değildir.

Interface örneği

```
public interface IDatabase {
    Book search(String name);
    int delete(String name);   // silinen kitap sayısını döndürür
    void insert(Book b);
}

public class OracleDB implements IDatabase {
    public Book search(String name) { System.out.println("Oracle searching"); return null; }
    public int delete(String name)  { System.out.println("Oracle deleting");  return 0; }
    public void insert(Book b)      { System.out.println("Oracle insert"); }
}

// Test
IDatabase r = new OracleDB();      // ileride: new MsSqlDatabase();
r.delete("Kitap");
```

Enum örneği (slayttan)

```
public enum Tariff {
    NETFREE(0, 4, 60), NET4(4, 8, 30), NET6(6, 8, 40);   // ilk satırda olmalı
    private int quota, speed, fee;
    private Tariff(int quota, int speed, int fee) {
        this.quota = quota; this.speed = speed; this.fee = fee;
    }
    public int getQuota() { return quota; }
    public int getSpeed() { return speed; }
    public int getFee()   { return fee; }
}
```

- `Tariff t = Tariff.NET4;` yazılır. Her sabit, constructor'a verilen değerlerle oluşturulmuş bir nesnedir.
- Örnekte `Person` ve `Subscription` arasında birebir association vardır. Her üyeliğin bir tarifesi ve oluşturulma tarihi vardır.

Generic sınıf

```
public class GenericClass<T> {
    private T data;
    public GenericClass(T data) { this.data = data; }
    public T write() { System.out.println("Ekrana yaz " + data); return data; }
}

GenericClass<Integer> g = new GenericClass<Integer>(5);
int f = g.write();                      // Ekrana yaz 5
GenericClass<String> s = new GenericClass<String>("aa");
String t = s.write();                   // dönüş tipi String olur
```

- `GenericClass<Book>` için `write` kitabın `toString` metodunu kullanır. Bu yüzden `Book`'a `toString` eklendi.

Ders sonrası notlar

- **Interface metotları da soyuttur.** Slayt anlatımında "bunlar soyut metot değil" dendi. Gövdesiz interface metotları örtük olarak `public abstract`'tır. Fark şudur: Soyut sınıfta `abstract` yazmak gerekir, interface'te gerekmez. Interface alanları da örtük olarak `public static final`'dır.
- **Interface'e eklenebilenler:** Java 8'den beri `default` ve `static` metotlar, Java 9'dan beri `private` metotlar yazılabilir.
- **C++'ta çoklu kalıtım vardır.** Hoca önce "C++'ta da yok diye biliyorum" dedi, sonra ekranda arayıp var olduğunu gösterdi. Sınıflar arası çoklu kalıtım C++ ve Python'da var, Java ve C#'ta yok.
- **"Bu adresler hiçbir zaman aynı olamaz" ifadesi yanlış.** Bir öğrenci `c1 == c2` için böyle dedi. İki referans aynı nesneyi gösteriyorsa `== true` döner. Hoca da bunu gösterdi.
- **`ArrayList` bağlı liste değildir.** Slayttaki "single linked" ifadesi yanlış. `ArrayList` arka planda yeniden boyutlanan bir dizi kullanır. Thread-safe değildir (bu kısım doğru). Bağlı liste `LinkedList`'tir ve Java'da çift bağlıdır.
- **Kapasite artışı:** OpenJDK'da yeni kapasite yaklaşık `eski + eski/2` olur (1,5 kat). `new ArrayList<>()` başta boş bir dizi kullanır, ilk eklemede kapasite 10 olur.
- **Java enum'larının sıra numarası vardır.** Slaytta "C'nin aksine karşılık gelen tamsayı yok" deniyor. Enum sabitleri tamsayı değildir ama `ordinal()` metodu sabitin sırasını (0'dan başlayarak) verir.
- **Enum constructor'ı:** `public` veya `protected` yazılamaz. Erişim belirteci yazılmasa da örtük olarak private'tır.

- **Generic sözdizimi:** Slaytta “büyük eşit sembolü” diye geçiyor. Doğrusu açılı parantezdir: `<T>`. `T` bir tip parametresidir. C’deki `#define` gibi metin değiştirme yapmaz. Derleme zamanında tip denetimi sağlar.
- **İkili ağaçta arama:** Ağaç dengeliyse $O(\log n)$’dir. Dengesiz bir ağaç en kötü durumda bağlı listeye döner ve $O(n)$ olur.
- **Python karşılaştırma metotları:** `__lt__` “less than” demektir. Derste “lower than” dendi.
- **Hocanın `IDatabase r = new OracleDB();` satırında aldığı hata** IDE’nin geç derlemesinden kaynaklandı. “Build” sonrası düzeldi. Kodda hata yoktu.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.